CEG 4424/6424 Security Attacks and Defenses
Homework #4

Student Name: ____Utsav Acharya_____
Wright UID: _____U01113272_____
Undergraduate/Graduate: _____UG_____

Question 1 (10 points) Modern operating systems have incorporated a bit "non_executable" to label whether the content in a memory page (e.g., a memory block) can be executed. If stack frames are organized into these memory pages and labeled as "non_executable".

(1) (5 points) Will this stop the shellcode **injected to the stack** (e.g., the buffer allocated in the stack) from be executed? Please justify.

Answer:

Yes , this will stop the shellcode injected to the stack from be executed.

Justification: The Non-Executable (NX/XD) bit prevents the CPU from executing code instructions stored in memory regions designated as data storage, such as the stack and heap. Since traditional buffer overflow attacks rely on injecting shellcode directly into a stack buffer and jumping the instruction pointer (`EIP`) to it, the CPU will raise a memory protection fault (Segmentation Fault) and halt execution when it attempts to run instructions from a non-executable page.

(2) (5 Points, Question for Graduate Students) Will this eradicate the possibility that an attacker executes unwanted code in the target system?

Answer:

NO, this will not eradicate the possibility that an attacker executes unwanted code in the target system.

Justification: While NX/XD stops direct execution of injected shellcode on the stack or heap, it does not prevent code reuse attacks such as **Return-Oriented Programming (ROP)** or **Jump-Oriented Programming (JOP)**. In a ROP attack, the attacker does not inject new code; instead, they chain together small snippets of existing, legitimate executable code already present in memory (called "gadgets," ending in a `ret` instruction) to achieve arbitrary malicious execution while fully respecting non-executable memory protections.

Question 2 (10 points) We use a 32-bit operating system. Its int types, memory addresses, and registers, such as ESP, EBP and EIP, are represented by 32 bits. We compile a program that calls this function in this operating system.

```
int foo(int x) {

    int y, z;
    y = x+3;
    z = x * 2;
    return z;

}
```

Please plot the stack frame of foo function before it returns.

Answer

Stack Frame Layout of foo Before It Returns (Top to Bottom / High Address to Low Address):

Memory Address / Offset	**Content**	**Size**	**Description**
High Addresses ($\uparrow$)	`x` (parameter)	4 bytes	Function parameter passed by caller
	Return Address (`EIP`)	4 bytes	Stored instruction pointer to return to caller
	Saved Frame Pointer (`EBP`)	4 bytes	Previous stack frame pointer
Low Addresses ($\downarrow$)	Local variable `y`	4 bytes	Local variable storage (`y = x + 3`)
	Local variable `z`	4 bytes	Local variable storage (`z = x * 2`)
Current `ESP` $\rightarrow$	*(Top of Stack)*	—	Stack pointer pointing to current top

Question 3 (20 points) We use a 32-bit operating system. Its int types, memory addresses, and registers, such as ESP, EBP and EIP, are represented by 32 bits. We compile a program that calls this function in this operating system.

```
void foo(int **input, int par){
        int tag = par;
        int *arr[10];
        printf("%x", tag);
        if(tag > 1){
           while(*input != 0x0000)
           {
              *arr = *input;
```

```
            arr++;
            input++;
        }
    }
    printf("%x", tag);
}
```

(1) (10 points)Please plot the stack frame of foo function before it returns.
Ansswer":
Stack Frame Layout of foo:

Memory Address / Offset	**Content**	**Size**	**Description**
High Addresses ($\uparrow$)	`par`	4 bytes	Parameter `par`
	`input`	4 bytes	Parameter `input` pointer
	Return Address (`EIP`)	4 bytes	Saved return address
	Saved Frame Pointer (`EBP`)	4 bytes	Saved frame pointer
	Local variable `tag`	4 bytes	Local variable storing `par`
	Local array `arr[0]` to `arr[9]`	40 bytes	Local array of 10 integer pointers (10×4 bytes)
Low Addresses ($\downarrow$)	*(Top of Stack / `ESP`)*	—	Current stack pointer

(2) (10 Points, Question for Graduate Students) Suppose you design a buffer overflow attack that successfully launches a shell by exploiting the foo function, after this successful BOF attack, will the two "printf" commands yield the same output? Please justify.

Question 4 (10 points) Alice can read the file A, can read and write the file B, and cannot access the file C. Bob can read and write the file A, can execute the file B, and can read the file C.

1. Write a set of access control lists for this situation.

An ACL is associated with each **object** (file), specifying which subjects have access:

- **File A:** Alice: `r`, Bob: `r,w`
- **File B:** Alice: `r,w`, Bob: `execute` (or `x`)
- **File C:** Alice: `none`, Bob: `r`

2. Write a set of capability lists for this situation.

   Answer:

A capability list is associated with each **subject** (user), specifying what objects they can access and how:

- **Alice:** File A (`r`), File B (`r,w`)
- **Bob:** File A (`r,w`), File B (`execute`), File C (`r`)

Question 5 (20 points)

What is ransomware? Please give the basic introduction of three types of ransomware after reading the paper (The answers to this question should be at least half a page. You should summarize the content in your own words.)

ANswer:

Introduction to Ransomware:

Ransomware is a malicious type of software that infiltrates a user's computer or network, targets their critical files or operating system, and encrypts them using robust cryptographic algorithms (such as AES and public-key RSA). The attackers then demand a ransom—typically payable in cryptocurrency like Bitcoin within a strict deadline—in exchange for the decryption key, operating effectively as a multi-million-dollar cybercriminal enterprise.

Three Types of Ransomware: Based on the variety of attacks and behaviors documented in security literature and malware research:

1. **File-Encrypting Ransomware (Crypto-Ransomware):** This type specifically targets and encrypts user files and documents (such as photos, databases, and office files) using strong encryption while leaving the core operating system intact, rendering individual files inaccessible until a ransom is paid (e.g., WanaCry, CryptoLocker, BitCrypt).
2. **Master Boot Record (MBR) / Disk-Encrypting Ransomware:** This aggressive variant infects the computer's master boot record or partition table, blocking the normal operating system from booting up entirely and displaying a ransom demand directly on a pre-boot lock screen (e.g., Petya).
3. **Locker / Screen-Locker Ransomware:** This form locks down the user interface or desktop entirely with a graphical full-screen window demanding payment, often without necessarily encrypting every file on the hard drive individually, preventing the user from interacting with any applications or system settings.